

Introduction to Databases

Assessment Questions — Answers

Q1. Explain the key differences between SQL and NoSQL databases with examples.

SQL (Relational Databases) store data in structured tables with predefined schemas. They use Structured Query Language (SQL) and enforce ACID properties (Atomicity, Consistency, Isolation, Durability). Relationships between tables are maintained using foreign keys.

Examples: MySQL, PostgreSQL, Oracle DB.

NoSQL (Non-Relational Databases) store data in flexible formats — documents, key-value pairs, wide columns, or graphs. They do not require a fixed schema and are designed for horizontal scaling and high-throughput workloads.

Examples: MongoDB (document), Redis (key-value), Cassandra (column), Neo4j (graph).

Key Differences:

- **Schema:** SQL has a fixed schema; NoSQL is schema-flexible or schema-less.
- **Scaling:** SQL scales vertically (bigger machine); NoSQL scales horizontally (more machines).
- **Transactions:** SQL supports full ACID; NoSQL often follows BASE (Basically Available, Soft state, Eventually consistent).
- **Query Language:** SQL uses standard SQL; NoSQL uses database-specific APIs.
- **Use Case:** SQL suits structured, relational data (banking, ERP); NoSQL suits unstructured or rapidly changing data (social media, IoT).

Q2. What does the CAP theorem state, and why can't a distributed system guarantee all three properties?

The **CAP Theorem**, introduced by Eric Brewer, states that a distributed data system can only guarantee two out of the following three properties simultaneously:

- **Consistency (C):** Every read receives the most recent write or an error. All nodes in the system see the same data at the same time.
- **Availability (A):** Every request receives a response (not an error), though it may not contain the most recent data.

- **Partition Tolerance (P):** The system continues to operate even when network partitions occur (i.e., messages between nodes are lost or delayed).

Why can't all three be guaranteed together?

In any real distributed system, network partitions (P) are unavoidable — hardware failures and network issues will always exist. When a partition happens, a system must choose: either stop responding to maintain consistency (sacrifice Availability), or keep responding with possibly stale data (sacrifice Consistency). You cannot have a consistent AND always-available system when a partition cuts communication between nodes.

This is why distributed systems are categorized as CP (e.g., HBase, Zookeeper) or AP (e.g., CouchDB, Cassandra).

Q3. Describe three scenarios where MongoDB would be preferred over a relational database.

1. Content Management Systems (CMS): Blog posts, articles, and media pages have varying fields — some have tags, some have videos, others just text. MongoDB's document model lets you store each content type as a flexible JSON-like document without schema migrations every time a new field is added.

2. Real-Time Analytics and Event Logging: Applications like e-commerce platforms track millions of events per day (clicks, page views, purchases). MongoDB handles high-volume writes efficiently and can store heterogeneous event objects without forcing a rigid table structure.

3. Catalog and Product Management: An e-commerce store sells electronics, clothing, and groceries — each with completely different attributes. In SQL, this requires either many nullable columns or complex EAV (Entity-Attribute-Value) patterns. In MongoDB, each product is simply a document with only its relevant fields, making the data model clean and intuitive.

Q4. Why does MongoDB use BSON internally instead of storing documents as JSON?

MongoDB stores documents in **BSON (Binary JSON)** rather than plain JSON for several important reasons:

1. Richer Data Types: JSON natively supports only strings, numbers, arrays, objects, booleans, and null. BSON extends this with types like Date, ObjectId, Decimal128, Binary, Int32, Int64, and Regular Expression — all essential for real-world database applications.

2. Efficient Encoding and Traversal: BSON is a binary format, which means it is more compact and faster to encode/decode than plain text JSON. MongoDB can also traverse BSON documents quickly without parsing the entire string, since BSON stores field

lengths, enabling efficient skipping.

3. ObjectId for Unique Identifiers: BSON introduces the ObjectId type — a 12-byte identifier that encodes a timestamp, machine ID, and counter. This allows MongoDB to generate globally unique `_id` values without a central sequence generator.

4. Ordered Key-Value Pairs: BSON preserves field order, which matters for index efficiency and certain query operations.

In short, BSON gives MongoDB the flexibility of JSON with the performance and type-safety needed for a production-grade database engine.

Q5. Write MongoDB queries to find all students with GPA above 3.5 and enrolled in 'CS101'.

Using the MongoDB shell or Mongoose in Node.js:

```
// MongoDB Shell Query
db.students.find({
  gpa: { $gt: 3.5 },
  enrolledCourses: "CS101"
})

// With projection (show only name, gpa, enrolledCourses)
db.students.find(
  { gpa: { $gt: 3.5 }, enrolledCourses: "CS101" },
  { name: 1, gpa: 1, enrolledCourses: 1, _id: 0 }
)

// Using Mongoose (Node.js)
const results = await Student.find({
  gpa: { $gt: 3.5 },
  enrolledCourses: "CS101"
});
console.log(results);
```

The query uses the **\$gt** (greater than) comparison operator to filter GPA, and matches documents where the **enrolledCourses** array contains the string 'CS101'. MongoDB automatically checks array membership when querying array fields with a scalar value.